Module 09 - Object-Oriented Programming Advanced

Table of Contents

- [ ] 1. Encapsulation — Protecting Your Data
- [ ] 2. Properties — Controlled Access with `@property`
- [ ] 3. Inheritance — Reusing and Extending Classes
- [ ] 4. `super()` — Talking to the Parent
- [ ] 5. Method Overriding
- [ ] 6. Static Methods — Utility Functions Inside a Class
- [ ] 7. Class Methods — Factory Patterns
- [ ] 8. Summary and Next Steps

---

1. Encapsulation — Protecting Your Data

Encapsulation means controlling who can read or modify an object's data. Think of it like access control in Active Directory: not everyone should be able to change everything.

In C++ and Java, access modifiers (`public` , `protected` , `private`) are enforced by the compiler. Python takes a different philosophy — **convention over enforcement**. As the documentation says: "*We are all adults here.*"

Python has three visibility levels, all simulated by naming conventions:

Prefix	Level	Convention	Enforced?
No prefix	Public	Anyone can read/write	No
`_single`	Protected	Only this class and subclasses	No — convention only
`__double`	Private	Only this class	Yes — via name mangling

Imagine a `Session` object that holds a user's authentication token. If `token` is public, any part of the codebase could accidentally overwrite it. Making it private forces all changes to go through controlled methods.

In [ ]:
```
# Topic: Public attributes — the default in Python
# Anyone can read or modify public attributes freely

class Session:
```

```
    """Represents an authenticated user session."""

    def __init__(self, username, token, privilege_level):
        self.username = username           # public — fine to read
        self.token = token                 # public — but anyone can overwrite!
        self.privilege_level = privilege_level  # public


session = Session("alice", "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9", "admin")

# Public access — no restriction, anyone can do this:
print(session.token)
session.token = "INJECTED_TOKEN"   # dangerous — nothing stops this
print(session.token)
```

In [ ]:

```
# Topic: Protected attributes — single underscore convention
# The underscore is a signal: 'internal use only, touch with care'
# It is NOT enforced — the code still runs if you access it directly

class Session:
    def __init__(self, username, token, privilege_level):
        self.username = username              # public — fine to expose
        self._token = token                   # protected — internal use
        self._privilege_level = privilege_level  # protected

    def get_privilege(self):
        """Controlled read access to privilege level."""
        return self._privilege_level


session = Session("alice", "eyJhbGc...", "admin")

# The underscore is a warning sign, not a lock:
print(session._token)   # works — but you're breaking the convention
print(session.get_privilege())  # correct way — through the method
```

In [ ]:

```
# Topic: Private attributes — double underscore (name mangling)
# Python internally renames __attr to _ClassName__attr
# This prevents accidental access from outside the class

class Session:
    def __init__(self, username, token, privilege_level):
        self.username = username
        self.__token = token                   # private — name-mangled
        self.__privilege_level = privilege_level  # private

    def validate_token(self, provided_token):
        """Compares token without ever exposing the real value."""
        return provided_token == self.__token  # internal access works fine

    def get_privilege(self):
        return self.__privilege_level


session = Session("alice", "eyJhbGc...", "admin")
```

```
# Direct access attempt:
try:
    print(session.__token)  # AttributeError!
except AttributeError as e:
    print(f"Access blocked: {e}")

# Correct way — through the method:
print(session.validate_token("eyJhbGc..."))  # True
print(session.validate_token("wrong_token")) # False

# Name mangling — Python internally renamed it:
print(session._Session__token)  # still accessible if you know the trick
# This is why Python says 'we are all adults here'
```

⚠️ Warning — Encapsulation in Python is Convention, Not a Security Control

Python's `__private` attributes **are not a security feature**. They prevent accidental access but can be bypassed with `_ClassName__attr`. Do not rely on them to protect sensitive data at runtime.

In real security tools, sensitive values (tokens, passwords, keys) must be protected by:

- Secure secret stores (HashiCorp Vault, AWS Secrets Manager)
- Environment variables — never hardcoded
- Proper authentication and authorisation at the API or OS level

Private attributes are for **good code design** — preventing bugs and enforcing interfaces.

🎯 Practice — Encapsulation

Exercise 1 — Write

Create a `Credential` class with:

- A public attribute `username`
- A private attribute `__password_hash` (store the value passed in)
- A method `verify_password(self, plain_text)` that returns `True` if the plain text matches the stored hash (for this exercise, just compare them as strings)
- A method `__str__` that shows the username but **never** the password hash

Then test that accessing `__password_hash` directly raises an `AttributeError`.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

2. Properties — Controlled Access with `@property`

Private attributes are good, but how do you let someone **read** the value safely? The `@property` decorator lets you expose an attribute as if it were public, while keeping full control over what happens when it is read or written.

```
@property
def attribute_name(self):
    return self.__attribute   # getter

@attribute_name.setter
def attribute_name(self, value):
    # validate before storing
    self.__attribute = value
```

The caller uses it like a normal attribute: `obj.cvss_score` — no parentheses. But behind the scenes, your getter/setter code runs.

In [ ]:
```
# Topic: @property — validation on write, clean read syntax

class Vulnerability:
    def __init__(self, cve_id, cvss_score):
        self.cve_id = cve_id
        self.__cvss_score = cvss_score   # private storage

    @property
    def cvss_score(self):
        """Getter — read the CVSS score safely."""
        return self.__cvss_score

    @cvss_score.setter
    def cvss_score(self, value):
        """Setter — validate before storing (CVSS is always 0.0-10.0)."""
        if not 0.0 <= value <= 10.0:
            raise ValueError(f"CVSS score must be 0.0-10.0, got {value}")
        self.__cvss_score = value

    def __str__(self):
        return f"{self.cve_id} — CVSS {self.cvss_score}"


vuln = Vulnerability("CVE-2023-44487", 7.5)
print(vuln.cvss_score)    # 7.5 — reads like a plain attribute

vuln.cvss_score = 8.2     # updates via the setter
print(vuln)

try:
    vuln.cvss_score = 15.0  # invalid — setter raises ValueError
except ValueError as e:
    print(f"Validation error: {e}")
```

---

3. Inheritance — Reusing and Extending Classes

Inheritance lets one class **inherit** all the attributes and methods of another class, then add or change what it needs.

Think of it like job roles in a SOC:

- Every `Analyst` has a name, employee ID, and can write reports
- A `SeniorAnalyst` has all of that, *plus* can approve escalations
- A `ThreatHunter` has all of that, *plus* can run hunting queries

```
Analyst (Parent / Base class)
  |
  +-- SeniorAnalyst (Child / Subclass)  -- inherits everything +
adds approve_escalation()
  |
  +-- ThreatHunter  (Child / Subclass)  -- inherits everything +
adds run_hunt()
```

Syntax:

```python
class ChildClass(ParentClass):   # <-- ParentClass in parentheses
    pass
```

In [ ]:

```python
# Topic: Basic inheritance — child class gets all parent behaviour

class Analyst:
    """Base class for all SOC team members."""

    def __init__(self, name, employee_id, team):
        self.name = name
        self.employee_id = employee_id
        self.team = team

    def write_report(self, incident_id):
        return f"{self.name} wrote report for incident {incident_id}"

    def __str__(self):
        return f"{self.name} (ID: {self.employee_id}) — Team: {self.team}"


class ThreatHunter(Analyst):   # <-- inherits from Analyst
    """Specialised analyst who proactively hunts for threats."""

    def run_hunt(self, hypothesis):
        """Threat hunters run queries based on attacker behaviour hypotheses."""
        return f"{self.name} running hunt: '{hypothesis}'"


hunter = ThreatHunter("Sarah", "EMP-007", "Threat Intelligence")
```

```
# Inherited method from Analyst:
print(hunter.write_report("INC-2024-0042"))

# New method specific to ThreatHunter:
print(hunter.run_hunt("Living-off-the-land by insider threat"))

# __str__ also inherited:
print(hunter)
```

---

4. `super()` — Talking to the Parent

When a child class needs its **own** `__init__` (to add new attributes), it still needs to call the parent's `__init__` to set up the inherited attributes.

`super()` gives you a reference to the **parent class**, so you don't have to hardcode its name.

```
class ChildClass(ParentClass):
    def __init__(self, parent_param1, parent_param2, child_param):
        super().__init__(parent_param1, parent_param2)  # call parent
setup first
        self.child_param = child_param                  # then add child's
own stuff
```

In [ ]:

```
# Topic: super() in the child constructor

class Analyst:
    def __init__(self, name, employee_id, team):
        self.name = name
        self.employee_id = employee_id
        self.team = team

    def write_report(self, incident_id):
        return f"{self.name} wrote report for incident {incident_id}"

    def __str__(self):
        return f"{self.name} (ID: {self.employee_id}) — Team: {self.team}"


class SeniorAnalyst(Analyst):
    """Senior analyst with additional clearance level and approval rights."""

    def __init__(self, name, employee_id, team, clearance_level):
        super().__init__(name, employee_id, team)  # parent sets name, id, team
        self.clearance_level = clearance_level      # child adds clearance

    def approve_escalation(self, incident_id):
        """Senior analysts can officially escalate to IR team."""
        return (
            f"[L{self.clearance_level} APPROVED] {self.name} "
            f"escalated {incident_id} to IR team"
```

```
        )

    def __str__(self):
        base = super().__str__()        # reuse parent's __str__
        return f"{base} | Clearance: L{self.clearance_level}"


senior = SeniorAnalyst("Marcus", "EMP-003", "Blue Team", clearance_level=3)

print(senior)                                          # includes clearance level
print(senior.write_report("INC-2024-0099"))            # inherited method
print(senior.approve_escalation("INC-2024-0099"))    # child-specific method
```

🎯 Practice — Inheritance and `super()`

Exercise 2 — Write

The `NetworkDevice` base class is defined below. Create a `Firewall` subclass that:

- Adds two new attributes: `rule_count` (int) and `default_policy` (str: `'DENY'` or `'ALLOW'`)
- Uses `super()` to call the parent `__init__`
- Adds a method `is_secure_default(self)` that returns `True` if `default_policy` is `'DENY'` (deny-by-default is the secure configuration)
- Overrides `__str__` to include the firewall-specific info on top of the parent's string

In [ ]:
```
class NetworkDevice:
    """Base class for all network infrastructure devices."""

    def __init__(self, hostname, ip_address, os_version):
        self.hostname = hostname
        self.ip_address = ip_address
        self.os_version = os_version

    def __str__(self):
        return f"{self.hostname} ({self.ip_address}) running {self.os_version}"


# Create your Firewall class here:


fw = Firewall("fw-core-01", "10.0.0.1", "PAN-OS 11.1", rule_count=247, default_poli
print(fw)
print("Secure default:", fw.is_secure_default())  # True
```

▶ 💡 **SOLUTION**

---

5. Method Overriding

A child class can **override** any method from the parent by defining a method with the same name. When you call `child.method()`, Python uses the child's version, not the parent's.

This is how you specialise behaviour:

- The parent defines a generic implementation
- Each child replaces it with behaviour specific to its type

In [ ]:
```
# Topic: Method overriding — child replaces parent behaviour

class Analyst:
    def __init__(self, name, employee_id):
        self.name = name
        self.employee_id = employee_id

    def describe_role(self):
        return f"{self.name} is a cybersecurity analyst."


class PenTester(Analyst):
    """Offensive security specialist."""

    def describe_role(self):   # overrides the parent method
        return f"{self.name} is a penetration tester — finds vulnerabilities before


class IncidentResponder(Analyst):
    """Specialist who handles active security incidents."""

    def describe_role(self):   # overrides the parent method
        return f"{self.name} is an incident responder — contains and recovers from


generic = Analyst("Alex", "EMP-001")
tester = PenTester("Hana", "EMP-012")
responder = IncidentResponder("Tom", "EMP-019")

print(generic.describe_role())
print(tester.describe_role())
print(responder.describe_role())
# Each object uses its own version of describe_role()
```

🎯 Practice — Override

Exercise 3 — Write

The `Vulnerability` base class has a method `recommended_action(self)` that returns a generic string. Create a subclass `RansomwareVulnerability` that:

- Adds an attribute `ransomware_family` (str, e.g. `'LockBit'`)
- Overrides `recommended_action` to return a more specific, urgent message that includes the ransomware family name

Then create a regular `Vulnerability` and a `RansomwareVulnerability` and print the `recommended_action()` for each.

In [ ]:
```
class Vulnerability:
    def __init__(self, cve_id, cvss_score, affected_system):
        self.cve_id = cve_id
        self.cvss_score = cvss_score
        self.affected_system = affected_system

    def recommended_action(self):
        return f"Apply vendor patch for {self.cve_id} on {self.affected_system}."


# Create your RansomwareVulnerability subclass here:
```

▶ 💡 **SOLUTION**

---

6. Static Methods — Utility Functions Inside a Class

A **static method** is a function that lives inside a class but does **not** access any instance data (`self`) or class data (`cls`).

Use it when the logic is **related to the class conceptually**, but does not need any data from a specific object — typically a utility or helper function.

```
class MyClass:
    @staticmethod
    def utility_function(param):
        # no self, no cls — pure logic
        return result
```

You call it on the class directly: `MyClass.utility_function(value)` (or on an instance, though the class call is cleaner).

Method type	First param	Access to instance?	Access to class?
Regular method	`self`	Yes	Yes
Static method	Nothing	No	No
Class method	`cls`	No	Yes

In [ ]:
```
# Topic: @staticmethod — utility logic that belongs with the class
# but doesn't need any object's data

class Vulnerability:
    def __init__(self, cve_id, cvss_score, affected_system):
        self.cve_id = cve_id
        self.cvss_score = cvss_score
        self.affected_system = affected_system
```

```python
    @staticmethod
    def score_to_severity(score):
        """Convert a CVSS score to a severity label.
        Static because this logic doesn't depend on any specific vulnerability obje
        """
        if score >= 9.0:
            return "CRITICAL"
        elif score >= 7.0:
            return "HIGH"
        elif score >= 4.0:
            return "MEDIUM"
        else:
            return "LOW"

    @staticmethod
    def is_valid_cve_format(cve_id):
        """Validates CVE format: CVE-YYYY-NNNNN
        Useful before creating an object — doesn't need self.
        """
        parts = cve_id.split("-")
        return (
            len(parts) == 3
            and parts[0] == "CVE"
            and parts[1].isdigit() and len(parts[1]) == 4
            and parts[2].isdigit()
        )


# Call on the class — no instance needed:
print(Vulnerability.score_to_severity(9.8))    # CRITICAL
print(Vulnerability.score_to_severity(3.1))    # LOW

print(Vulnerability.is_valid_cve_format("CVE-2023-44487"))  # True
print(Vulnerability.is_valid_cve_format("MS-2023-001"))     # False

# Also works on an instance (but calling on the class is cleaner):
vuln = Vulnerability("CVE-2023-44487", 9.8, "nginx")
print(vuln.score_to_severity(7.5))  # HIGH
```

🎯 Practice — Static Methods

Exercise 4 — Write

Add two static methods to the `NetworkPort` class:

1. `is_valid_port(port_number)` — returns `True` if the port number is between 1 and 65535
2. `get_risk_level(port_number)` — returns:
   - `'HIGH'` if the port is in `[21, 23, 3389, 5900]` (FTP, Telnet, RDP, VNC — high risk)
   - `'MEDIUM'` if the port is in `[80, 3306, 1433]` (HTTP, MySQL, MSSQL — medium risk)

- `'LOW'` for everything else

Test both methods.

In [ ]:
```
class NetworkPort:
    def __init__(self, port_number, service_name, is_open):
        self.port_number = port_number
        self.service_name = service_name
        self.is_open = is_open

    # Add your static methods here


# Test:
print(NetworkPort.is_valid_port(443))      # True
print(NetworkPort.is_valid_port(99999))    # False
print(NetworkPort.get_risk_level(23))      # HIGH
print(NetworkPort.get_risk_level(3306))    # MEDIUM
print(NetworkPort.get_risk_level(443))     # LOW
```

▶ 💡 **SOLUTION**

---

7. Class Methods — Factory Patterns

A **class method** has access to the **class itself** (not a specific instance). It receives `cls` as its first parameter instead of `self`.

The most common use case is the **factory pattern** — creating objects from alternative input formats.

```
class MyClass:
    @classmethod
    def from_something(cls, data):
        # parse data, then create and return an instance
        return cls(arg1, arg2)
```

In security tools, this is extremely useful for parsing log lines or JSON API responses directly into objects.

In [ ]:
```
# Topic: @classmethod as a factory — create objects from log strings

class SecurityAlert:
    """Represents a SIEM alert."""

    def __init__(self, alert_id, severity, source_ip, rule_name):
        self.alert_id = alert_id
        self.severity = severity
        self.source_ip = source_ip
        self.rule_name = rule_name
```

```python
    @classmethod
    def from_log_line(cls, log_line):
        """Parse a raw log string and return a SecurityAlert object.
        Log format: 'ALT-0042|HIGH|185.220.101.47|SSH_BRUTE_FORCE'
        """
        parts = log_line.split("|")
        return cls(
            alert_id=parts[0],
            severity=parts[1],
            source_ip=parts[2],
            rule_name=parts[3]
        )

    def __str__(self):
        return (
            f"[{self.severity}] {self.alert_id} | "
            f"{self.rule_name} from {self.source_ip}"
        )


# Create from normal constructor:
alert_a = SecurityAlert("ALT-0001", "CRITICAL", "10.0.0.99", "MALWARE_DETECTED")

# Create from a raw log line (e.g. reading from a SIEM export file):
raw_log = "ALT-0042|HIGH|185.220.101.47|SSH_BRUTE_FORCE"
alert_b = SecurityAlert.from_log_line(raw_log)

print(alert_a)
print(alert_b)
```

---

8. Summary and Next Steps

Quick Reference

Concept	What it does	When to use it
`_single`	Protected — convention, not enforced	Internal attributes/methods
`__double`	Private — name mangled, harder to access	Sensitive data, enforce interface
`@property`	Getter/setter with validation	Controlled access to private attrs
`class Child(Parent)`	Inheritance	Reuse and extend behaviour
`super()`	Call parent's methods	Extend parent `__init__` or methods
Override	Define same method in child	Specialise behaviour per type
`@staticmethod`	No `self`, no `cls`	Utility/helper logic

Concept	What it does	When to use it
`@classmethod`	Receives `cls`	Factory constructors

Next Steps

- **02_Drills_OOP.ipynb** — 15 exercises to practice all these patterns
- After OOP: **Exception Handling** — `try` / `except`, custom exceptions (common in security tools)